

AI Usage Documentation

This section documents all AI interactions undertaken during the development of this assignment. AI tools were used in four phases: architecture decision-making, feature implementation, Apple HIG auditing, and visual bug diagnosis. All code generated by AI tools was reviewed, tested in the simulator, and in several cases rejected or substantially modified before use.

1.1 UI Mockup Generation – Claude Artifacts

All three UI mockup variations described in Section 2.5.2 were generated using Claude Artifacts at claude.ai. The complete prompts and outcomes are documented in Table 9 below. Mockup screenshots are presented in Figures 1–3.

Table 8: Complete AI Mockup Generation Prompt and Outcome Log

Variation	Tool	Full Prompt (Summarized)	AI Response Summary	Design Decision
1 – Minimalist Grid	Claude Artifacts (claude.ai)	Generate an iPadOS SwiftUI mockup for a childcare keyworker app. Include child list, diary entries, attendance status. Use clean minimal white design with card grid and bottom tab bar.	Produced a white-background card grid layout with circular avatars and bottom tab bar with five sections. Clean and readable but structured as an iPhone-first design.	Rejected as primary layout. Bottom tab bar is not appropriate for regular-class iPadOS. Card grid does not leverage iPad horizontal space. Aesthetic quality retained as reference.
2 – Sidebar Professional	Claude Artifacts (claude.ai)	Redesign as NavigationSplitView layout with persistent sidebar. Child rows with attendance badge and diary dot. Deep teal accent. Add detail panel for message and incident content.	Produced a NavigationSplitView mock with compact sidebar rows, color-coded attendance indicators, and a teal accent system. Correct iPad layout pattern.	Layout adopted. Color palette revised, deep teal judged too clinical for childcare. Sidebar structure, column widths, and compact row card design carried into implementation.
3 – EYFS Warm Palette	Claude Artifacts (claude.ai)	Third variation with warm EYFS color palette (coral, sage, sky-blue), rounded corners, large tap targets, high-contrast text labels, Dock shortcut row at bottom.	Produced a warm, rounded design with coral-green accents, oversized tap targets, and a Dock-style action row. Accessibility compliance clearly demonstrated.	Color palette, rounded corners, and accessibility decisions adopted. Dock pattern rejected, unfamiliar iPadOS paradigm. Warmth reinterpreted through nursery green accent and rounded card corners in final design.

1.2 Code Generation – Claude (claude.ai)

1. Phase 1 – Architecture Decisions

High-level architectural questions were posed to Claude covering three topics: the split-view layout approach, the messaging data model, and GDPR pattern design.

Prompt: I am building an iPadOS childcare management app with a three-panel workspace (sidebar, content, detail). Should I use NavigationSplitView or a custom ZStack-based layout? What are the trade-offs?

AI Response (summarized): NavigationSplitView is strongly preferred because it handles sidebar show/hide animation natively, integrates with the system keyboard shortcut Ctrl+Shift+S, provides the columnVisibility API for programmatic control, and receives future Apple enhancements automatically. The ZStack approach requires manually reimplementing these behaviors and is unlikely to remain consistent across iPadOS updates.

Action: NavigationSplitView adopted. The custom ZStack prototype was discarded. The recommendation was cross-validated against WWDC24 Session 10147 [1] before implementation.

2. Phase 2 – Performance and Memory Optimization

Prompt (Core Data Fetch Request Overhead): The AnalyticsDataService runs multiple @FetchRequest calls on the same view. The app feels sluggish when switching between children in the sidebar. How do I diagnose and reduce Core Data fetch overhead in a SwiftUI context?

AI Response (summarized): The likely cause is unbounded fetch requests with no fetchLimit or fetchBatchSize, combined with fetches firing on every view re-render. Recommendations: add fetchBatchSize: 20 to all list fetches, add fetchLimit where only a fixed window of records is needed (e.g., last 7 days), use NSSortDescriptor with an indexed attribute so Core Data can short-circuit the sort, and move @FetchRequest properties to dedicated child views rather than a single parent so only the relevant fetch re-fires when state changes.

Action: fetchBatchSize and fetchLimit were added to the mood trend and message sparkline fetches. The analytics dashboard was refactored, so each chart view owns its own @FetchRequest rather than deriving from a shared parent fetch. Sidebar switching latency was eliminated after these changes.

Prompt (FaceBlurEditorView Memory Pressure): The app crashes on older iPad models when a keyworker selects a high-resolution photo for a milestone entry. The Vision face detection and Core Image blur pipeline is processing the full-resolution image. How should I cap memory usage without degrading blur quality?

AI Response (summarized): Downsample the image before passing it to VNDetectFaceRectanglesRequest. A maximum input dimension of 1600 pt is sufficient for face detection accuracy; Vision's bounding boxes are normalized so they scale correctly back to any output size. Also normalize EXIF orientation before downsampling — orientation-mismatched bounding boxes are a common bug when skipping this step. Apply blur to the downsampled image, not the original; the difference is imperceptible at milestone photo display sizes.

Action: A pre-processing step was added to FaceBlurEditorView that caps the input image at 1600 pt and applies UIImage orientation normalisation before the Vision request. Memory pressure on low-end iPad models was resolved. The blurredFaceCount stored in Core Data

reflects detections on the downsampled image, which was verified to match the original in the MilestonePhotoPrivacyTests suite.

Prompt (SwiftUI Unnecessary Re-renders): Charts in ChildAnalyticsDashboardView are re-animating every time I update any unrelated state in the parent view. How do I prevent this?

AI Response (summarized): The .animation modifier without a value: parameter responds to any state change on the view, not just the relevant one. Replace .animation(.spring()) with .animation(.spring(), value: specificStateProperty) so the animation only triggers when that property changes. For charts, also ensure @FetchRequest properties are on the chart view itself rather than a parent — a fetch result change on a parent triggers a full subtree re-render.

Action: All .animation calls were audited and converted to the value-scoped form, as referenced in WWDC23 Session 10156 [2]. Unrelated state changes in KeyworkerIPadCoordinator no longer cause chart animations to re-fire.

3. Phase 3 – Apple HIG Audit

Prompt: Act as a senior iOS engineer auditing the NurseryConnect application for Apple Human Interface Guidelines compliance. Identify the 10 highest-priority violations and improvements, with justification for each.

AI Response (summarized): The audit returned a prioritized list of 10 items: convert custom button styles to .borderedProminent/.bordered, migrate fonts to Dynamic Type semantic styles, replace custom empty states with ContentUnavailableView, use presentationDetents on all sheets, add .hoverEffect to sidebar rows, enforce a single accent color, add pull-to-refresh to lists, apply .searchable to child list, add keyboard shortcuts, and use correct presentation modifiers.

Action: All 10 identified items were implemented. The Dynamic Type migration in particular required visual review of every screen (see Section 5.6).

4. Phase 4 – Bug Diagnosis

Prompt (visionOS Sheet Race Condition): My visionOS attendance check-in sheets are presenting empty. I have @State var selectedRecord: NCAttendanceRecord? and @State var showsCheckInSheet: Bool. Attaching code and screenshot. What is causing this?

AI Diagnosis: This is a classic SwiftUI presentation race condition. The sheet is presented when showsCheckInSheet becomes true, but selectedRecord may not yet have propagated. Replace both state variables with .sheet(item: \$checkInRecord) where checkInRecord is an optional NCAttendanceRecord. The optional itself serves as the gate.

Action: .sheet(item:) pattern adopted throughout both targets. Race condition eliminated completely in testing.

5. Rejected AI Outputs and Critical Review

Several AI-generated code patterns were identified as incorrect and rejected during the mandatory review phase:

- A recursive ButtonStyle.makeBody implementation that called .buttonStyle() on its own content internally. This is invalid in SwiftUI because button styles cannot call other button styles recursively. The style was rewritten as an opacity-only pass-through.

- An initialization call `KeyworkerChildSummary(from: child)` using an initializer that did not exist in the codebase. Accepted without verification, this would have caused a compile-time error. The correct construction path was identified from the model definition and used instead.
- The `summaryPill(label:count:color:Any)` function signature using an `Any`-typed color parameter. This caused cascading Swift type inference failures in surrounding `@ViewBuilder` closures, producing misleading compiler error messages pointing to unrelated lines. The function was rewritten with an explicit `Color` parameter and the `ContentUnavailableView` extracted into a separate computed property.

These cases demonstrate that AI-assisted development requires a higher critical review standard, not a lower one. AI tools generate syntactically plausible code very quickly, but plausibility is not correctness. All generated code must be understood, tested, and independently verified before integration. The developer's domain expertise, architectural judgment, and code review skills remain essential and non-delegable.